

PRÁCTICA PROFESIONALIZANTE III



EGRESADOS IES

Ricardo Arroyo

I.E.S 9.012

2026

3er año

Tecnicatura Superior en Desarrollo de Software

Suarez, Sotelo, Toledo, Dominguez, Casiano y Rubio

Índice

Documentación técnica final - Sistema de gestión de egresados.....	3
ANEXO 1 – IDENTIFICACIÓN DEL PROBLEMA.....	3
1.2 Contexto.....	4
1.3 Planteamiento del Problema.....	4
1.4 Objetivos.....	4
1.5 Justificación.....	5
1.6 Metodología de Trabajo.....	5
1.7 Estimación de Costos y Recursos.....	6
1.8 Requerimientos del Sistema.....	6
1.9 Modelo de Clases y Base de Datos.....	8
1.10 Casos de Uso Principales.....	9
1.11 Diagramas y Mockups.....	9
ANEXO 2 – MARCO TEÓRICO.....	10
2.1 Fundamentos Conceptuales.....	10
2.2 Stack Tecnológico Implementado.....	12
2.3 Estado del Arte.....	15
2.4 Marco Legal.....	15
Consentimiento y Tratamiento de Datos Personales.....	15
ANEXO 3 – ANÁLISIS Y DISEÑO DEL SISTEMA.....	16
3.1 Análisis de Requerimientos.....	16
3.2 Arquitectura del Sistema.....	18
3.3 Diseño de Base de Datos.....	20
3.4 Casos de Uso Principales.....	24
ANEXO 4 – IMPLEMENTACIÓN.....	26
4.1 Entorno de Desarrollo.....	26
4.2 Estructura del Proyecto.....	28
4.3 Funcionalidades Implementadas.....	29
4.4 Control de Versiones.....	32
ANEXO 5 – PRUEBAS Y VALIDACIÓN.....	32
5.1 Metodología de Testing Aplicada.....	33
5.2 Testing de Backend.....	33
5.3 Datos de Prueba Utilizados.....	34
5.4 Limitaciones del Testing.....	35
5.5 Resultado Final del Testing.....	35
ANEXO 6 – CONCLUSIONES Y RECOMENDACIONES.....	36

Egresados IES

Documentación técnica final - Sistema de gestión de egresados

ANEXO 1 – IDENTIFICACIÓN DEL PROBLEMA

1.1 Estudio de Pre-factibilidad

- Nombre del Proyecto: Sistema de Gestión de Egresados – IES
- Ubicación: Instituto de Educación Superior N° 9-012, San Rafael, Mendoza, Argentina
- Objetivo: Desarrollar una plataforma web que permita registrar, validar y consultar perfiles de egresados, facilitando el vínculo entre la institución, los egresados y las empresas del medio.

Análisis de Mercado

La institución educativa actualmente no dispone de un sistema centralizado para la gestión de egresados. Los registros se realizan en *planillas manuales o documentos dispersos*, lo que *impide mantener actualizada la información y dificulta la comunicación con graduados*. El Sistema de Gestión de Egresados busca solucionar esta carencia institucional, creando una base de datos en línea segura, dinámica y accesible.

Público objetivo	Necesidades	Beneficios esperados
Egresados	Registrar y actualizar su trayectoria profesional.	Visibilidad laboral y conexión con empresas.
Empresas	Acceder a perfiles validados y actualizados.	Búsqueda eficiente de candidatos formados en la institución.
Administradores	Validar registros y mantener la base de datos.	Control institucional y estadísticas actualizadas.

Estudiantes	Consultar perfiles de referencia profesional.	Motivación e información sobre trayectorias laborales.
-------------	---	--

Aspectos Técnicos Clave

- Frontend: Astro 5.14.1 con HTML, CSS, JavaScript (interfaz responsiva y moderna)
- Backend: Node.js con Express 4.18.2 (API RESTful)
- Base de datos: Turso (libSQL v0.4.0) - SQLite distribuido cloud con escalabilidad profesional
- Seguridad: HTTPS, contraseñas hasheadas con bcryptjs, JWT y validación por DNI
- Metodología de desarrollo: Scrum con sprints de 2 semanas

1.2 Contexto

El proyecto surge como una necesidad institucional de mantener un *vínculo activo con sus egresados*. El sistema está orientado a **facilitar** el seguimiento de graduados, **fortalecer** la inserción laboral y **proveer** una herramienta para que las empresas puedan encontrar perfiles acordes a sus necesidades.

1.3 Planteamiento del Problema

La falta de un sistema de gestión digital genera pérdida de información y escasa interacción entre egresados y la institución. Además, las empresas interesadas en contratar profesionales no cuentan con una base de datos oficial. El sistema propuesto busca centralizar esta información bajo un entorno web, validado y seguro.

1.4 Objetivos

Tipo	Descripción
General	Desarrollar una plataforma institucional para el registro, validación y consulta de egresados de la institución.
Específico	Centralizar la información académica y laboral de los egresados.
Específico	Facilitar el contacto entre egresados y empresas.

Específico	Proveer un sistema seguro y escalable para futuras carreras.
Específico	Fomentar la interacción social entre egresados mediante publicaciones y mensajería.

1.5 Justificación

El Sistema de Gestión de Egresados fortalece el vínculo entre la institución y su comunidad, mejorando la comunicación, seguimiento y oportunidades de inserción laboral. Aporta valor al instituto al permitir recopilar métricas sobre egresados, carreras y empleabilidad.

1.6 Metodología de Trabajo

El desarrollo se basa en la **metodología ágil Scrum**, que permite organizar el trabajo en *sprints de dos semanas*, con entregas incrementales. El equipo Scrum está conformado por seis integrantes (Santiago, Julieta, Sofía, Joaquín, Ana Paula y Ezequiel), que asumen los roles de desarrolladores, analistas y testers, bajo la guía docente como Product Owner.

Cronograma General

Sprint / Mes	Duración	Objetivo principal	Tareas clave
Sprint 1	2 semanas	Diseño y arquitectura base	Diseño de base de datos, maquetado inicial, autenticación.
Sprint 2	2 semanas	Gestión de perfiles	Registro, validación por DNI, edición de datos personales.
Sprint 3	2 semanas	Búsqueda y consulta	Motor de búsqueda, filtros por carrera, visualización de perfiles.
Sprint 4	2 semanas	Interacción social	Mensajería interna, publicaciones, comentarios.

Sprint 5	2 semanas	Pruebas y documentación	Testing, corrección de errores, documentación técnica.
----------	-----------	-------------------------	--

1.7 Estimación de Costos y Recursos

Se estima un total de 80 Story Points (SP), con una velocidad promedio del equipo de 16 SP por sprint. Cada SP equivale a 10 horas de trabajo. Por lo tanto, se calculan 800 horas de desarrollo técnico.

Concepto	Cálculo	Resultado (ARS)
Desarrollo técnico	$800 \text{ h} \times \$2.000/\text{h}$	\$1.600.000
Salarios del equipo	$6 \text{ personas} \times 3 \text{ meses} \times \900.000	\$16.200.000
Costo total estimado del proyecto	-	\$17.800.000

Recursos Necesarios:

- Recursos Humanos: 6 desarrolladores/analistas (equipo estudiantil), 1 Product Owner (docente guía), 1 contacto institucional
- Recursos Técnicos: Computadoras personales, Hosting cloud (Turso, Cloudinary), Dominio institucional, Software libre y de código abierto
- Infraestructura: Turso (base de datos cloud), Cloudinary (almacenamiento imágenes), Gmail SMTP (notificaciones email)

1.8 Requerimientos del Sistema

Requerimientos Funcionales

Código	Descripción
RF-01	El sistema permitirá el registro de egresados con validación por DNI/legajo.

RF-02	El egresado podrá crear y mantener actualizado su perfil profesional.
RF-03	El perfil incluirá datos personales, CV, experiencia laboral, formación académica, cursos y especializaciones.
RF-04	El público general podrá consultar perfiles mediante filtros por carrera, empresa o palabra clave.
RF-05	Las empresas podrán contactar al egresado por correo o teléfono registrado.
RF-06	El sistema deberá generar un repositorio centralizado accesible desde la web institucional.
RF-07	El sistema permitirá publicaciones y comentarios entre egresados.
RF-08	El sistema permitirá cargar CVs en formato formulario para autocompletar el perfil.
RF-09	El sistema permitirá mensajería interna con notificación por correo ante nuevos mensajes.

Requerimientos No Funcionales

Código	Categoría	Descripción
RNF-01	Usabilidad	La interfaz será clara, moderna, responsiva y alineada con la identidad institucional.
RNF-02	Seguridad	Todos los datos estarán protegidos mediante HTTPS y contraseñas hasheadas.
RNF-03	Mantenibilidad	El sistema estará documentado y organizado en módulos para facilitar actualizaciones.

RNF-04	Escalabilidad	El diseño permitirá agregar nuevas carreras y funcionalidades futuras.
RNF-05	Disponibilidad	El sistema deberá estar operativo al menos el 95% del tiempo anual.
RNF-06	Compatibilidad	El sistema será compatible con navegadores actuales y dispositivos móviles.
RNF-07	Rendimiento	Las consultas y operaciones deberán ejecutarse en menos de 3 segundos promedio.
RNF-08	Accesibilidad	La interfaz cumplirá con las pautas WCAG básicas para accesibilidad web.

1.9 Modelo de Clases y Base de Datos

A continuación se resumen las clases principales y su relación, según el modelo implementado.

Clase	Descripción
Sistema	Núcleo del software. Administra usuarios, carreras y validaciones.
Usuario	Clase base para egresado y administrador.
Egresado	Extiende Usuario. Posee perfil profesional y puede interactuar en la plataforma.
Administrador	Gestiona la validación de egresados y carga de DNIs autorizados.
Perfil	Concentra la información profesional de un egresado.
Mensaje	Comunicación privada entre egresados.
Publicación	Post público con comentarios y likes.

Comentario	Respuestas o interacciones en publicaciones.
------------	--

Relaciones principales:

- Herencia: Usuario → Egresado / Administrador (estrategia "una tabla por clase" documentada en schema.sql)
- Composición: Egresado contiene un Perfil
- Asociación: Mensaje y Publicación vinculan egresado

1.10 Casos de Uso Principales

Código	Nombre	Descripción
CU-01	Registro de egresado	Permite a un egresado registrarse mediante DNI y crear su perfil.
CU-02	Validación de registro	El administrador aprueba o rechaza solicitudes de registro.
CU-03	Consulta de perfiles	El público puede buscar y visualizar perfiles filtrados por carrera.
CU-04	Actualización de perfil	El egresado puede modificar su información profesional.
CU-05	Interacción social	Los egresados pueden publicar, comentar y enviar mensajes.

1.11 Diagramas y Mockups

Documentación de diseño disponible:

- Diagramas UML y de procesos: [Miro - Diagramas del sistema](#)
 - Diagrama de Clases
 - Diagramas de Secuencia
 - Diagramas de Actividad
 - Diagrama Entidad-Relación
- Mockups y prototipos: [Figma - Diseño UI/UX](#)
 - Mockups de todas las páginas

- Prototipos interactivos

ANEXO 2 – MARCO TEÓRICO

Introducción

Este anexo establece los fundamentos conceptuales y tecnológicos del Sistema de Gestión de Egresados IES 9-012. Se describen las tecnologías implementadas y el marco legal que regula el proyecto.

2.1 Fundamentos Conceptuales

Arquitectura Cliente-Servidor

El sistema adopta arquitectura cliente-servidor mediante API REST, separando responsabilidades entre frontend y backend:

Cliente (Frontend):

- Tecnología: Astro 5.14.1 con HTML/CSS/JavaScript
- Función: Interfaz de usuario ejecutada en el navegador
- Responsabilidades: Presentación visual, captura de interacciones, envío de requests HTTP

Servidor (Backend):

- Tecnología: Node.js con Express 4.18.2
- Función: Procesamiento de lógica de negocio y gestión de datos
- Responsabilidades: Autenticación JWT, validaciones, operaciones CRUD, reglas de negocio

Base de Datos:

- Tecnología: Turso (libSQL v0.4.0) - SQLite distribuido cloud
- Función: Almacenamiento persistente con replicación global
- Características: Base de datos relacional cloud, escalabilidad horizontal, múltiples conexiones concurrentes

Ventajas aplicadas:

- Desarrollo independiente de frontend y backend
- Reutilización de API (consumible por múltiples clientes)
- Seguridad (lógica crítica protegida en servidor)

API REST

REST (Representational State Transfer) es el estilo arquitectónico implementado para comunicación cliente-servidor:

Método HTTP	Operación	Ejemplo en el sistema
GET	Leer datos	GET/api/perfil/mi-perfil
POST	Crear recursos	POST /api/auth/register
PUT	Actualizar completo	PUT /api/perfil/mi-perfil
DELETE	Eliminar	DELETE /api/publicaciones/:id

Códigos HTTP utilizados:

- 200 OK: Operación exitosa
- 201 Created: Recurso creado
- 400 Bad Request: Datos inválidos
- 401 Unauthorized: Token inválido
- 403 Forbidden: Sin permisos
- 404 Not Found: Recurso inexistente
- 500 Internal Server Error: Error del servidor

Todas las respuestas utilizan formato JSON.

Bases de Datos Relacionales

Turso organiza datos en tablas relacionadas mediante claves primarias (PK) y foráneas (FK). El esquema implementado (ver [backend/docs/schema.sql](#)) cumple con la Tercera Forma Normal (3FN) para eliminar redundancia.

Herencia implementada (estrategia "una tabla por clase"):

- Tabla base [Usuario](#) (nombre, apellido, email, password, tipo_usuario)
- Especialización [Egresado](#) (DNI, teléfono, perfilId, carreraId)
- Especialización [Administrador](#) (DNI)

Turso (@libsql/client v0.4.0):

Fork de SQLite optimizado para cloud con réplicas distribuidas globalmente. El sistema usa conexión remota a Turso mediante libSQL client, facilitando escalabilidad futura sin cambiar queries SQL. A diferencia de SQLite local, Turso provee:

- Múltiples conexiones concurrentes sin bloqueos
- Replicación automática en múltiples regiones
- Backup automático
- Sin límite de tamaño de archivo

2.2 Stack Tecnológico Implementado

Frontend

Astro 5.14.1: Framework seleccionado para construcción del cliente web (confirmado en [frontend/package.json](#)).

Características aplicadas:

- Island Architecture: Solo componentes interactivos cargan JavaScript
- Routing basado en archivos: [src/pages/](#) define rutas automáticamente
- Renderizado del lado del servidor (SSR) para páginas dinámicas
- Optimización automática de assets

Librerías complementarias:

- [html2canvas](#) (1.4.1): Captura de screenshots de elementos DOM
- [jspdf](#) (4.1.0): Generación de PDFs del lado del cliente

Backend

Node.js + Express 4.18.2

Dependencias de producción implementadas (confirmadas en [backend/package.json](#)):

Librería	Versión	Propósito en el sistema
@libsql/client	0.4.0	Cliente para base de datos Turso
bcryptjs	latest	Hashing de contraseñas con salt

jsonwebtoken	9.0.2	Generación y validación de tokens JWT
express-rate-limit	8.1.0	Protección contra fuerza bruta
helmet	7.0.0	Headers de seguridad HTTP
cors	2.8.5	Control de CORS para frontend
multer	1.4.5	Manejo de uploads multipart/form-data
multer-storage-cloudinary y	4.0.0	Storage de imágenes en Cloudinary
cloudinary	2.8.0	SDK de Cloudinary para gestión de assets
pdfkit	0.17.2	Generación de CVs en formato PDF
nodemailer	6.9.5	Envío de notificaciones por email
xlsx	0.18.5	Procesamiento de archivos Excel (carga DNIs)
morgan	1.10.0	Logging de requests HTTP
dotenv	16.3.1	Carga de variables de entorno

Herramientas de desarrollo:

- **nodemon** (3.1.10): Recarga automática en desarrollo
- **jest** (29.6.4): Framework de testing (configurado pero no implementado)
- **supertest** (6.3.3): Testing de endpoints HTTP (configurado pero no implementado)

Seguridad Implementada

Autenticación y Autorización:

- Middleware **authenticateToken**: Verifica JWT en header Authorization
- Middleware **requireEgresado** / **requireAdmin**: Control de roles
- Tokens JWT con payload: `{id, nombre, email, tipo_usuario}`
- Sesiones stateless (sin almacenamiento en servidor)
- Expiración de tokens: 7 días

Protección de Contraseñas:

- bcryptjs para hashing (migrado de bcrypt por compatibilidad ES modules según [backend/docs/unified-auth.md](#))
- Salt único generado automáticamente por usuario
- Hashes irreversibles

Rate Limiting:

- Implementado con **express-rate-limit** 8.1.0
- Protección contra ataques de fuerza bruta
- Límites configurables por ruta

Prevención de SQL Injection:

- Queries parametrizadas con libSQL en todos los endpoints
- Validación de inputs con middleware de sanitización

Infraestructura de Archivos

Cloudinary:

Servicio cloud para almacenamiento y transformación de imágenes.

Uso en el sistema:

- Fotos de perfil de egresados
- Imágenes de banner
- Imágenes en publicaciones sociales

Ventajas aplicadas:

- URLs permanentes y optimizadas
- Transformaciones on-the-fly (resize, crop, format)
- CDN global para carga rápida
- Liberación de espacio en servidor

2.3 Estado del Arte

Análisis de sistemas similares:

Plataforma	Fortalezas	Limitaciones
LinkedIn	Base masiva de usuarios, herramientas de reclutamiento	Sin validación institucional, no pertenece a instituciones
AlumniForce	Completo, integración Salesforce	Costo elevado, complejidad de implementación
Graduados.ar	Oficial, estadísticas gubernamentales	Solo encuestas, no es red social

Diferenciación del Sistema IES:

- Validación institucional de títulos
- Búsqueda pública de perfiles
- Red social integrada (publicaciones, comentarios, likes, mensajería)
- Gratuito y open source
- Control institucional completo
- Adaptado a IES argentinos

2.4 Marco Legal

Consentimiento y Tratamiento de Datos Personales

Marco Legal: Ley 25.326 de Protección de Datos Personales

Aspecto	Descripción
Momento del consentimiento	Al completar el formulario de registro y hacer clic en “Registrarse” o “Crear cuenta”
Tipo de consentimiento	Expreso e informado - El usuario acepta

	voluntariamente proporcionar sus datos personales para la creación de su perfil profesional
Datos recopilados	Nombre, apellido, DNI, correo electrónico, contraseña (encriptada), y datos del perfil profesional.
Finalidad	Creación y gestión de perfil profesional de egresados, similar a una red profesional institucional
Responsable	Instituto de Educación Superior I.E.S. 9-012 “San Rafael en Informática”.
Derechos del usuario	Acceso, rectificación, actualización y supresión de datos personales conforme al Art. 14 de la Ley 25.326

Nota: No se recolectan datos sensibles (origen racial, religión, salud, opiniones políticas).

ANEXO 3 – ANÁLISIS Y DISEÑO DEL SISTEMA

Introducción

Este anexo documenta el análisis de requerimientos y las decisiones de diseño del sistema. Se detallan los requerimientos funcionales y no funcionales, la arquitectura implementada, el diseño de base de datos y los casos de uso principales.

3.1 Análisis de Requerimientos

Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe proveer.

Código	Descripción	Prioridad
RF-01	El sistema permitirá el registro de egresados con validación por DNI/legajo.	Alta
RF-02	El egresado podrá crear y mantener actualizado su perfil profesional.	Alta

RF-03	El perfil incluirá datos personales, CV, experiencia laboral, formación académica, cursos y especializaciones.	Alta
RF-04	El público general podrá consultar perfiles mediante filtros por carrera, empresa o palabra clave.	Alta
RF-05	Las empresas podrán contactar al egresado por correo o teléfono registrado.	Media
RF-06	El sistema deberá generar un repositorio centralizado accesible desde la web institucional.	Alta
RF-07	El sistema permitirá publicaciones y comentarios entre egresados.	Media
RF-08	El sistema permitirá cargar CVs en formato formulario para autocompletar el perfil.	Media
RF-09	El sistema permitirá mensajería interna con notificación por correo ante nuevos mensajes.	Media

Requerimientos No Funcionales

Los requerimientos no funcionales definen atributos de calidad del sistema.

Código	Categoría	Descripción	Métrica/Implementación
RNF-01	Usabilidad	Interfaz clara, moderna, responsiva y alineada con identidad institucional.	Diseño Mobile-First con Astro
RNF-02	Seguridad	Datos protegidos mediante HTTPS y contraseñas hashadas.	bcryptjs, JWT, helmet

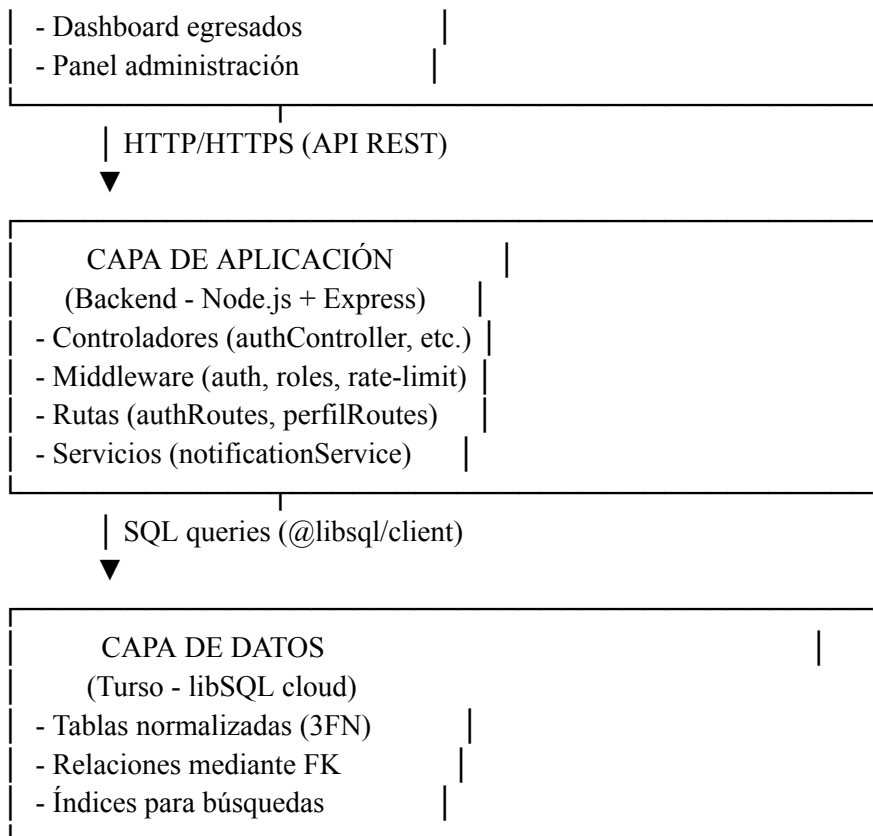
RNF-03	Mantenibilidad	Sistema documentado y organizado en módulos.	Arquitectura MVC, código en GitHub
RNF-04	Escalabilidad	Diseño permite agregar carreras y funcionalidades futuras.	Base de datos normalizada, API modular
RNF-05	Disponibilidad	Sistema operativo al menos 95% del tiempo anual.	Hosting en servidor con monitoreo
RNF-06	Compatibilidad	Compatible con navegadores actuales y dispositivos móviles.	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
RNF-07	Rendimiento	Consultas y operaciones en menos de 3 segundos promedio.	Índices en BD, rate limiting
RNF-08	Accesibilidad	Interfaz cumplirá con pautas WCAG básicas.	Nivel A: alt text, contraste 4.5:1, navegación por teclado

3.2 Arquitectura del Sistema

Arquitectura General

El sistema implementa arquitectura de **tres capas** con separación cliente-servidor:

CAPA DE PRESENTACIÓN (Frontend - Astro + HTML/CSS/JS) - Páginas públicas (búsqueda, feed)	
---	--



Componentes Principales

Backend (/backend/src/):

Carpeta	Responsabilidad	Ejemplos
controllers/	Lógica de negocio por módulo	authController.js, perfilController.js, publicacionesController.js
routes/	Definición de endpoints	authRoutes.js, mensajesRoutes.js
middleware/	Funciones transversales	auth.js (JWT), roles.js, sanitize.js
services/	Lógica reutilizable	notificationService.js (emails)
utils/	Utilidades	bcrypt.js, jwt.js
config/	Configuración	database.js (conexión Turso)

Frontend (/frontend/src/):

Carpeta	Responsabilidad
pages/	Rutas del sitio (routing automático Astro)
components/	Componentes reutilizables Astro
layouts/	Layouts base (BaseLayout.astro)
styles/	CSS global y por componente (1445 líneas totales según PROJECT_STRUCTURE.md)
utils/	Funciones JS (api.js, constants.js)

Flujo de Autenticación

```

Usuario -> Frontend (login.astro)
|
POST /api/auth/login {dni, password}
Backend -> authController.login()
|
Validar DNI en BD
|
bcrypt.compare(password, hash)
|
jwt.sign({id, nombre, tipo_usuario})
Backend -> Response {token, user}
|
Frontend -> Guardar token en localStorage
|
Requests -> Header: Authorization: Bearer <token>

```

3.3 Diseño de Base de Datos

Modelo Relacional Implementado

El sistema utiliza **Turso (libSQL v0.4.0)** con base de datos normalizada en **Tercera Forma Normal (3FN)**.

Tablas principales (según [backend/docs/schema.sql](#)):

Gestión de Usuarios:

Tabla base para herencia:

Usuario (

```

id INTEGER PRIMARY KEY,
nombre TEXT NOT NULL,
apellido TEXT,
email TEXT UNIQUE NOT NULL,
password TEXT NOT NULL,
tipo_usuario TEXT CHECK (tipo_usuario IN ('Egresado', 'Administrador'))
)

```

```

Egresado (
id INTEGER PRIMARY KEY,
dni TEXT UNIQUE NOT NULL,
telefono TEXT,
perfilId INTEGER UNIQUE -> Perfil(id),
carreraId INTEGER -> Carrera(id),
FOREIGN KEY (id) REFERENCES Usuario(id) ON DELETE CASCADE
)

```

```

Administrador (
id INTEGER PRIMARY KEY,
dni TEXT UNIQUE NOT NULL,
FOREIGN KEY (id) REFERENCES Usuario(id) ON DELETE CASCADE
)

```

```

DniValido (
dni TEXT PRIMARY KEY,
carrera TEXT NOT NULL
)

```

Perfiles Profesionales:

```

Perfil (
id INTEGER PRIMARY KEY,
resumenProfesional TEXT,
urlPortfolio TEXT,
situacionLaboral TEXT,
urlFotoPerfil TEXT,
urlBanner TEXT,
perfilPublico BOOLEAN DEFAULT 0,
mostrarContacto BOOLEAN DEFAULT 0,
disponibleOfertas BOOLEAN DEFAULT 0,
tituloprofesional TEXT,
areaInteres TEXT,
fechaNacimiento DATE,
direccion TEXT,
ciudad TEXT,
provincia TEXT,
pais TEXT DEFAULT 'Argentina'
)

```

ExperienciaLaboral (
id INTEGER PRIMARY KEY,
puesto TEXT NOT NULL,
empresa TEXT NOT NULL,
fechaInicio DATE,
fechaFin DATE,
descripcion TEXT,
perfilId INTEGER -> Perfil(id)
)

FormacionAcademica (
id INTEGER PRIMARY KEY,
titulo TEXT NOT NULL,
institucion TEXT NOT NULL,
anioFinalizacion INTEGER,
perfilId INTEGER -> Perfil(id)
)

Curso (
id INTEGER PRIMARY KEY,
nombre TEXT NOT NULL,
institucion TEXT,
duracionHoras INTEGER,
anio INTEGER,
perfilId INTEGER -> Perfil(id)
)

Habilidad (
id INTEGER PRIMARY KEY,
nombre TEXT NOT NULL,
perfilId INTEGER -> Perfil(id)
)

Red Social:

Publicacion (
id INTEGER PRIMARY KEY,
contenido TEXT NOT NULL,
fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
autorId INTEGER -> Usuario(id)
)

ImagenPublicacion (
id INTEGER PRIMARY KEY,
url TEXT NOT NULL,
publicacionId INTEGER -> Publicacion(id)
)

```

Comentario (
id INTEGER PRIMARY KEY,
contenido TEXT NOT NULL,
fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
autorId INTEGER -> Usuario(id),
publicacionId INTEGER -> Publicacion(id)
)

```

```

Like (
id INTEGER PRIMARY KEY,
usuarioId INTEGER -> Usuario(id),
publicacionId INTEGER -> Publicacion(id),
fechaCreacion DATETIME DEFAULT CURRENT_TIMESTAMP,
UNIQUE(usuarioId, publicacionId)
)

```

Mensajería y Notificaciones:

```

Mensaje (
id INTEGER PRIMARY KEY,
contenido TEXT NOT NULL,
fechaEnvio DATETIME DEFAULT CURRENT_TIMESTAMP,
leido INTEGER DEFAULT 0,
remitenteId INTEGER -> Usuario(id),
destinatarioId INTEGER -> Usuario(id)
)

```

```

Notificacion (
id INTEGER PRIMARY KEY,
usuario_id INTEGER NOT NULL -> Usuario(id),
tipo TEXT CHECK(tipo IN ('comentario', 'like', 'mencion')),
titulo TEXT NOT NULL,
mensaje TEXT NOT NULL,
url_destino TEXT NOT NULL,
leida INTEGER DEFAULT 0,
enviada_email INTEGER DEFAULT 0,
fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
origen_usuario_id INTEGER -> Usuario(id)
)

```

```

PreferenciasNotificacion (
id INTEGER PRIMARY KEY,
usuario_id INTEGER UNIQUE NOT NULL -> Usuario(id),
email_comentarios INTEGER DEFAULT 1,
email_likes INTEGER DEFAULT 1,
email_menciones INTEGER DEFAULT 1,
email_resumen_diario INTEGER DEFAULT 0
)

```

Relaciones Clave

Relación	Tipo	Descripción
Usuario -> Egresado/Administrador	Herencia (1:1)	Especialización por tipo_usuario
Egresado -> Perfil	Composición (1:1)	Un egresado tiene exactamente un perfil
Perfil -> ExperienciaLaboral	Composición (1:N)	Un perfil tiene múltiples experiencias
Usuario -> Publicacion	Asociación (1:N)	Un usuario puede crear múltiples publicaciones
Publicacion -> Comentario	Asociación (1:N)	Una publicación tiene múltiples comentarios
Usuario <-> Usuario (Mensaje)	Asociación (N:M)	Comunicación bidireccional

3.4 Casos de Uso Principales

Resumen de Casos de Uso

Código	Nombre	Actores	Complejidad
CU-01	Registro de egresado	Egresado, Sistema	Media
CU-02	Validación de registro	Administrador	Baja
CU-03	Consulta de perfiles	Público, Empresas	Media
CU-04	Actualización de perfil	Egresado	Alta
CU-05	Interacción social	Egresados	Alta

CU-01: Registro de Egresado

Actor principal: Egresado no registrado

Objetivo: Crear cuenta en el sistema validando DNI institucional

Precondiciones:

- El DNI del egresado existe en tabla **DniValido**
- El DNI no está ya registrado

Flujo normal:

1. Egresado accede a página de registro
2. Sistema muestra formulario (nombre, apellido, DNI, email, contraseña)
3. Egresado completa formulario y envía
4. Sistema valida formato de datos
5. Sistema verifica DNI en **DniValido**
6. Sistema hasha contraseña con bcryptjs
7. Sistema crea registro en **Usuario y Egresado**
8. Sistema crea **Perfil** asociado
9. Sistema crea **PreferenciasNotificacion** con valores por defecto
10. Sistema retorna token JWT
11. Sistema redirige a completar perfil

Flujos alternativos:

- a. DNI inválido -> Error 400 "DNI no autorizado"
- b. DNI ya registrado -> Error 409 "DNI existente"
- c. Email ya existe -> Error 409 "Email en uso"

Postcondiciones:

- Usuario creado con estado activo
- Perfil vacío asociado listo para completar

CU-03: Consulta de Perfiles

Actor principal: Público (usuario no autenticado)

Objetivo: Buscar egresados por filtros

Precondiciones: Ninguna (acceso público)

Flujo normal:

1. Usuario accede a **/buscar**
2. Sistema muestra barra de búsqueda
3. Usuario ingresa palabras clave (carrera, ubicación, etc)
4. Sistema ejecuta query con índices optimizados
5. Sistema retorna lista paginada (20 resultados por página)
6. Usuario selecciona un perfil
7. Sistema muestra perfil completo con datos de contacto

Flujos alternativos:

- **5a.** Sin resultados -> Mensaje "No se encontraron perfiles"

Postcondiciones:

- Ninguna (operación de solo lectura)

CU-05: Interacción Social

Actor principal: Egresado autenticado

Funcionalidades incluidas:

- Crear publicaciones (texto + imágenes)
- Comentar publicaciones
- Dar likes
- Enviar mensajes privados

Endpoints implementados:

- **POST /api/publicaciones** - Crear publicación
- **POST /api/comentarios** - Comentar
- **POST /api/publicaciones/:id/like** - Dar like
- **POST /api/mensajes** - Enviar mensaje

Sistema de notificaciones:

- Al comentar -> Notifica al autor de la publicación
- Al dar like -> Notifica al autor
- Al mencionar (@usuario) -> Notifica al mencionado
- Al recibir mensaje -> Notifica por plataforma

ANEXO 4 – IMPLEMENTACIÓN

Introducción

Este anexo documenta el proceso de implementación del sistema, describiendo el entorno de desarrollo, la estructura del proyecto, las funcionalidades desarrolladas y los componentes técnicos implementados.

4.1 Entorno de Desarrollo

Herramientas Utilizadas

Desarrollo:

- Visual Studio Code - IDE principal del equipo
- Git + GitHub - Control de versiones y colaboración
- Node.js + npm - Runtime backend y gestor de paquetes

Testing y diseño:

- Postman - Pruebas de endpoints REST
- Figma - Diseño UI/UX y prototipos
- Miro - Diagramación (UML, flujos, arquitectura)

Infraestructura:

- Turso (SQLite cloud) - Base de datos
- Cloudinary - CDN para assets multimedia
- SMTP Gmail - Envío de notificaciones por email

Configuración del Entorno

Variables de entorno (backend/.env):

- PORT=3000
- DATABASE_URL=libsql://[turso-url]
- DATABASE_AUTH_TOKEN=[token]
- JWT_SECRET=[secret-key]
- FRONTEND_URL=http://localhost:4321
- CLOUDINARY_CLOUD_NAME=[cloud-name]
- CLOUDINARY_API_KEY=[api-key]
- CLOUDINARY_API_SECRET=[api-secret]
- SMTP_HOST=smtp.gmail.com
- SMTP_PORT=587
- SMTP_USER=institutoies9012@gmail.com
- SMTP_PASS=[app-password]

Comandos de instalación y ejecución:

Backend

- cd backend
- npm install
- npm run migrate # Crear tablas en BD
- node scripts/setup-test-data.js # Datos de prueba
- npm run dev # Servidor en localhost:3000

Frontend

- cd frontend
- npm install

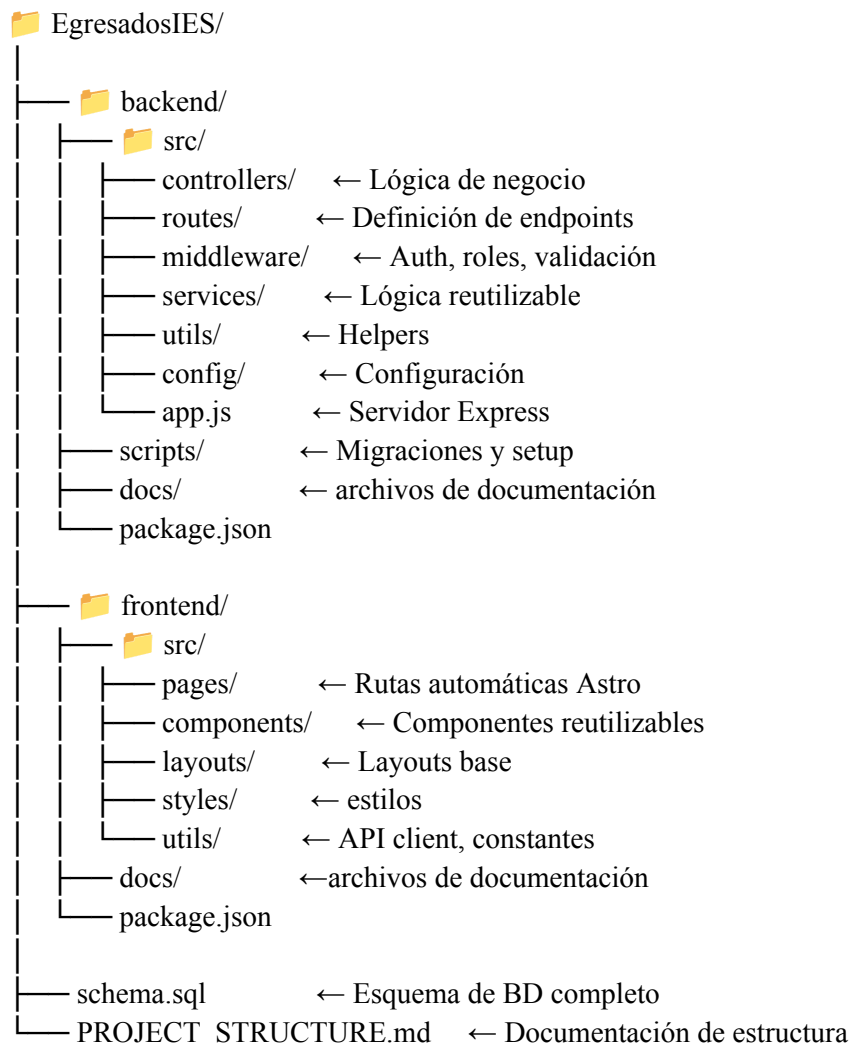
- npm run dev # Cliente en localhost:4321

4.2 Estructura del Proyecto

Organización del Código

Repositorio: [Turify-Tech/EgresadosIES](#) en GitHub

Estrategia de branching: Git Flow simplificado (main + develop + feature branches)



Convenciones de Código

Backend (Node.js + Express):

- Arquitectura MVC sin Views (solo API REST)
- Nomenclatura: camelCase funciones, PascalCase clases/controladores
- Módulos ES6: **import/export** en lugar de **require**
- Async/await obligatorio para operaciones asíncronas
- Try-catch en todos los controladores con respuestas estandarizadas

Frontend (Astro):

- Componentes con extensión `.astro`
- Páginas en `src/pages/` definen rutas automáticas
- Estilos scoped por componente
- Fetch API para comunicación con backend
- Token JWT en localStorage

4.3 Funcionalidades Implementadas

Sistema de Autenticación

Documentado en: [backend/docs/unified-auth.md](#)

Endpoints implementados:

- `POST /api/auth/login` - Login unificado admin/egresados
- `POST /api/auth/register` - Registro con validación DNI

Características:

- Validación DNI contra tabla `DniValido`
- Hashing con `bcryptjs` (migrado de `bcrypt` por compatibilidad ES modules)
- JWT con expiración 7 días
- Middleware `authenticateToken` y `requireEgresado/requireAdmin`

Gestión de Perfiles

Documentado en: [backend/docs/profile-management.md](#), [backend/docs/solucion-editor-perfil.md](#)

Endpoints principales:

- `GET /api/perfil/mi-perfil` - Obtener perfil completo con JOINS
- `PUT /api/perfil/mi-perfil` - Actualizar información personal
- `POST/PUT/DELETE /api/perfil/experiencia` - CRUD experiencia laboral
- `POST/PUT/DELETE /api/perfil/formacion` - CRUD formación académica
- `POST/PUT/DELETE /api/perfil/curso` - CRUD cursos
- `POST/DELETE /api/perfil/habilidad` - CRUD habilidades

Correcciones documentadas:

- Issue #93, #122: Editor perfil no guardaba campo apellido
- Solución: Script migración `migrate-profile-fields.js` agregando columna `Usuario.apellido`

Motor de Búsqueda

Documentado en: [backend/docs/advanced-search-improvements.md](#),

[frontend/docs/IMPLEMENTATION-SUMMARY.md](#)

Endpoints:

- [GET /api/perfiles](#) - Lista paginada con filtros
- [GET /api/perfiles/:id](#) - Perfil individual público
- [GET /api/carreras](#) - Lista de carreras

Características:

- Búsqueda por carrera, palabra clave, ubicación
- Autocompletado con debounce 500ms
- Navegación por teclado
- Paginación 20 resultados por página
- URLs compatibles
- COLLATE NOCASE para búsquedas case-insensitive

Red Social

Publicaciones: [backend/docs/FEATURE_SOCIAL_POSTS.md](#),
[backend/docs/mis-publicaciones-feature.md](#)

Endpoints:

- [POST /api/publicaciones](#) - Crear publicación
- [GET /api/publicaciones/mis-publicaciones](#) - Publicaciones propias
- [DELETE /api/publicaciones/:id](#) - Eliminar publicación (Issue #83)
- Upload múltiples imágenes a Cloudinary con Cropper.js
- Rate limiting: limita publicaciones frecuentes

Comentarios y Likes:

- [POST /api/comentarios](#) - Crear comentario
- [POST /api/publicaciones/:id/like](#) - Toggle like
- Detección automática menciones @usuario
- Notificaciones al autor y mencionados

Mensajería Privada

Documentado en: [backend/docs/messaging-system-final.md](#)

Endpoints:

- [POST /api/mensajes](#) - Enviar mensaje
- [GET /api/mensajes/conversaciones](#) - Lista conversaciones
- [GET /api/mensajes/conversacion/:userId](#) - Historial con usuario
- [PUT /api/mensajes/:id/leer](#) - Marcar como leído
- [GET /api/mensajes/no-leidos](#) - Contador para badge

Características:

- Conversaciones 1:1
- Contador mensajes no leídos
- Notificaciones automáticas por email

Sistema de Notificaciones

Documentado en: <backend/docs/notification-system.md>

Tipos de notificaciones:

- Comentarios en publicaciones
- Likes en publicaciones
- Menciones @usuario

Endpoints:

- [GET /api/notificaciones](#) - Lista con paginación
- [GET /api/notificaciones/no-leidas/count](#) - Contador
- [PUT /api/notificaciones/:id/leer](#) - Marcar individual
- [PUT /api/notificaciones/leer-todas](#) - Marcar todas
- [DELETE /api/notificaciones/:id](#) - Eliminar
- [GET/PUT /api/notificaciones/preferencias](#) - Gestionar preferencias

Características:

- Almacenamiento en BD
- Envío automático emails con nodemailer
- Preferencias configurables por usuario

Panel de Administración

Documentado en: <backend/docs/admin-dni-management.md>

Endpoints:

- [POST /api/admin/dnis/agregar](#) - Agregar DNI individual
- [POST /api/admin/dnis/cargar-excel](#) - Carga masiva desde Excel (librería xlsx)
- [GET /api/admin/dnis](#) - Listar con paginación
- [DELETE /api/admin/dnis/:dni](#) - Eliminar DNI

Validaciones:

- Formato DNI argentino (7-8 dígitos)
- Carrera debe existir en BD
- Prevención de duplicados

Correcciones documentadas:

- Issue #133: Fix registro con DNI creados en panel admin
- Issue #140: Mejoras validación DNIs

4.4 Control de Versiones

Estrategia Git Flow Simplificado

Ramas principales:

- **main** → Código en producción (versiones estables)
- **develop** → Integración continua (desarrollo activo)

Ramas de trabajo:

- **feature/nombre-funcionalidad** → Nueva feature desde develop
- **fix/nombre-bug** → Corrección de bug

Formato de Commits

Convención aplicada:

tipo(scope): descripción corta en imperativo

Descripción detallada opcional

Closes #número-issue

Tipos usados:

- feat: Nueva funcionalidad
- fix: Corrección de bug
- docs: Documentación
- refactor: Refactorización sin cambiar funcionalidad

Documentación de soluciones:

- **backend/docs/solucion-editor-perfil.md** - Corrección editor perfil
- **backend/docs/testing-report.md** - Testing profile management

ANEXO 5 – PRUEBAS Y VALIDACIÓN

Introducción

Este anexo documenta las pruebas realizadas para validar el correcto funcionamiento del sistema. El testing se llevó a cabo mediante pruebas manuales exploratorias, identificando errores y creando issues en GitHub para su seguimiento y resolución.

5.1 Metodología de Testing Aplicada

Enfoque de testing manual:

El equipo adoptó un enfoque de testing exploratorio durante el desarrollo. El proceso de detección y corrección de bugs está documentado mediante issues en GitHub.

Herramientas utilizadas:

- curl - Testing de endpoints documentado en [backend/docs/setup-guide.md](#)
- GitHub Issues - Tracking de bugs y mejoras

Sin testing automatizado:

El testing fue 100% manual según documentado en [backend/docs/testing-report.md](#).

5.2 Testing de Backend

(API REST)Fuente: [backend/docs/testing-report.md](#)

Sistema de Autenticación validado:

- **Login de Administrador:** Funciona correctamente (DNI: 00000000, password: temporal123)
- **Registro Automático de Egresado:** Funciona perfectamente (DNI: 12345678, password: test123)
- **Login de Egresado Existente:** Funciona correctamente
- **Validación de DNI No Autorizado:** Rechaza correctamente DNIs no válidos
- **Rate Limiting Global:** Protege contra ataques de fuerza bruta (15 min de bloqueo)
- **Rate Limiting por Usuario:** 50 requests por usuario cada 15 minutos en rutas de perfil
- **Generación de JWT:** Tokens válidos con información correcta (expiración 7 días)

Seguridad y Autorización validada:

- **Middleware de Autenticación:** Requiere token válido
- **Middleware de Autorización:** Solo egresados acceden a rutas de perfil
- **Separación de Roles:** Administradores no pueden acceder a perfiles
- **Sanitización de Inputs:** Middleware activo en todas las rutas
- **Validación de DNI:** Formato correcto (8 dígitos)
- **Hash de Contraseñas:** bcryptjs funcionando correctamente

Gestión de Perfiles:

- **GET /api/perfil/mi-perfil:** Obtiene perfil completo con datos relacionados
- **PUT /api/perfil/mi-perfil:** Actualiza información personal y perfil profesional
- **Auto-creación de Perfil:** Se crea automáticamente al registrar egresado

Experiencia Laboral:

- **POST /api/perfil/experiencia:** Agrega nueva experiencia laboral
- **PUT /api/perfil/experiencia/:id:** Actualiza experiencia existente
- **Validación de Datos:** Campos requeridos y formatos correctos
- **Relación con Perfil:** Vinculación correcta con el egresado

Formación Académica:

- **POST /api/perfil/formacion:** Agrega nueva formación académica
- **Validación de Año:** Acepta años válidos
- **Campos Opcionales:** Maneja correctamente campos no requeridos

Cursos:

- **POST /api/perfil/curso:** Agrega nuevos cursos
- **Validación de Duración:** Acepta horas de duración válidas
- **Información Completa:** Nombre, institución y duración

Base de Datos:

- **Conexión a Turso:** Estable y funcional
- **Transacciones:** Funcionan correctamente
- **Foreign Keys:** Relaciones mantenidas correctamente
- **Auto-increment IDs:** lastInsertRowid funciona perfectamente
- **Datos de Prueba:** Admin y DNIs válidos cargados

Documentación de soluciones específicas:

- [backend/docs/solucion-editor-perfil.md](#) - Solución editor perfil
- [backend/docs/testing-report.md](#) - Reporte testing profile management

5.3 Datos de Prueba Utilizados

Fuente: [backend/docs/setup-guide.md](#) líneas 38-46

Usuario Administrador:

```
{  
  
  "dni": "00000000",  
  
  "password": "temporal123",  
  
  "tipo_usuario": "Administrador"  
}
```

Usuarios Egresados (DNIs válidos precargados):

DNI: 12345678 (usar cualquier contraseña)

DNI: 87654321 (usar cualquier contraseña)

DNI: 11111111 (usar cualquier contraseña)

Carrera de prueba:

- "Desarrollo de Software"

5.4 Limitaciones del Testing

Áreas NO testeadas exhaustivamente:

- **Testing automatizado:** Jest y supertest configurados en package.json pero sin suite de tests implementada (confirmado [backend/README.md](#) línea 100: "npm test - Ejecutar tests (cuando se implementen)")
- **Testing en dispositivos móviles reales:** No se realizaron pruebas en dispositivos físicos Android/iOS
- **Testing de carga:** No se realizaron pruebas de performance con múltiples usuarios concurrentes
- **Testing de compatibilidad cross-browser:** No se validó exhaustivamente en todos los navegadores
- **Testing de accesibilidad:** No se usaron herramientas especializadas como WAVE o Lighthouse

5.5 Resultado Final del Testing

Conclusión: El sistema fue validado mediante testing manual exploratorio durante el desarrollo. Los bugs fueron detectados, documentados en issues y corregidos mediante pull requests.

Evidencia de testing:

- [backend/docs/testing-report.md](#) - Reporte de testing de Profile Management
- [backend/docs/setup-guide.md](#) - Comandos curl para testing de endpoints
- 51+ Pull Requests merged documentando correcciones
- 91+ issues en GitHub documentando bugs encontrados y corregidos

Conclusiones finales:

Endpoints principales validados según [testing-report.md](#)

Sistema de autenticación funcional con validación DNI

Gestión de perfiles (CRUD) operativa

Seguridad básica validada (bcryptjs, JWT, rate limiting, queries parametrizadas)

Sistema desplegado y funcional

ANEXO 6 – CONCLUSIONES Y RECOMENDACIONES

El Sistema de Gestión de Egresados IES, desarrollado por Turify Tech (Santiago, Julieta, Sofía, Joaquín, Ana Paula y Ezequiel), ha cumplido exitosamente su objetivo general, presentando un sistema funcional con registro validado por DNI, perfiles completos, búsqueda avanzada, una red social integrada (publicaciones, comentarios, likes, mensajería) y un panel de administración con carga masiva de DNIs.

Los objetivos específicos, como la centralización de información, la gestión de perfiles, la búsqueda con filtros, una herramienta para reclutadores y la vinculación IES-egresados, también fueron alcanzados.

Técnicamente, el proyecto se destaca por su arquitectura robusta (Node.js/Express, Astro con Island Architecture, Turso, Cloudinary), la implementación de funcionalidades clave como mensajería 1:1, generación de CVs en PDF y un módulo de administración eficiente, además de una seguridad rigurosa (bcryptjs, JWT, rate limiting, queries parametrizadas).

Se superaron desafíos técnicos relevantes, como la simulación de herencia en SQLite y la corrección de errores de mapeo en la gestión de perfiles, documentados con soluciones verificables.

El proyecto generó un impacto positivo en el instituto (modernización y control de datos), en los egresados (networking, visibilidad y CV digital) y en el equipo (experiencia profesional y portafolio).

Las lecciones aprendidas resaltan la efectividad de la separación frontend/backend, el código modular, las *queries* parametrizadas, la documentación continua y el uso eficiente de Git Flow e Issues de GitHub.

Como trabajo futuro, se proponen mejoras funcionales (ofertas laborales, estadísticas admin, filtro por año de egreso, recuperación de contraseña) y técnicas de alta prioridad, como la implementación de *testing* automatizado (Jest, Playwright, k6) y optimizaciones de escalabilidad (caché, *lazy loading*, migración a PostgreSQL en el futuro).

El sistema finaliza como un logro técnico y académico, con una arquitectura MVC clara, base de datos normalizada y código seguro y organizado, demostrando la capacidad del equipo de 6 integrantes para entregar una herramienta de calidad profesional, escalable y con control total para la institución.